VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Database Management Systems (23CS3PCDBM)

Submitted by

Sarthak Thrishuleshwar (1BM24CS260)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2025 to Jan-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Database Management Systems (23CS3PCDBM)" carried out by **Sarthak Thrishuleshwar** (1BM24CS260), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025. The Lab report has been approved as it satisfies the academic requirements in respect of a Database Management Systems (23CS3PCDBM) work prescribed for the said degree.

Lab faculty Incharge Name Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

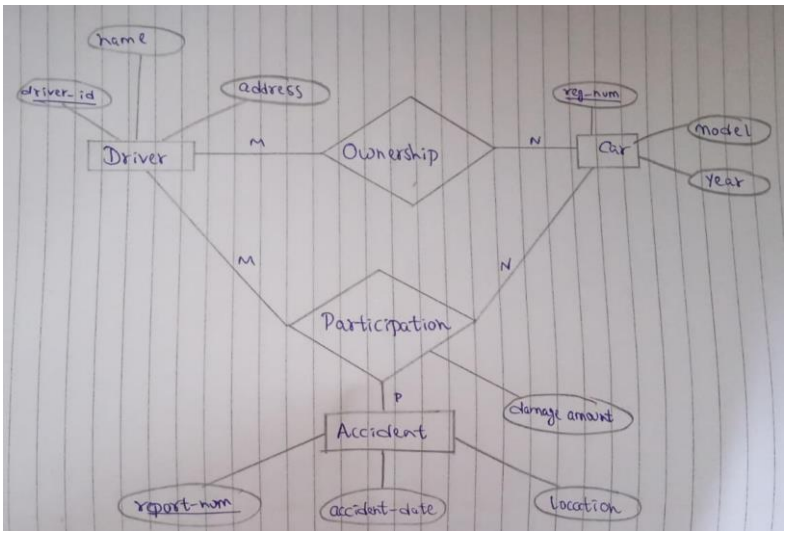
SI No	Date	Experiment Title	Page No.
1	22/09/2025	Insurance Database	4
2	29/09/2025	More Queries on Insurance Database	12
3	06/10/2025	Bank Database	15
4	13/10/2025	More Queries on Bank Database	23
5	3/11/2025	Employee Database	26
6	3/11/2025	More Queries on Employee Database	35
7	10/11/2025	Supplier Database	38
8	24/11/2025	More Queries on Supplier Database	46
8	8/12/2025	NOSQL Installation in Cloud	51
9	15/12/2025	NO SQL - Student Database	52
10	22/12/2025	NO SQL - Customer Database	55
11	22/12/2025	NO SQL – Restaurant Database	58
12	3/11/2025	LeetCode Practice	64
13	3/11/2025	LeetCode Practice	68

Experiment 1: Insurance Database

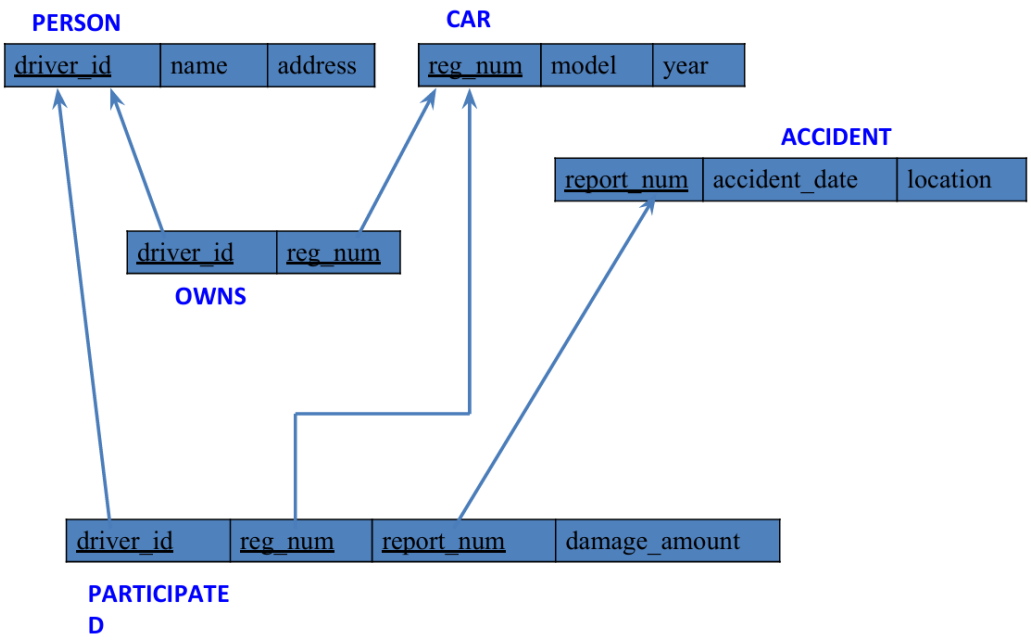
Specification of Insurance Database Application

The insurance database must maintain information about drivers, the cars they own, the accidents reported, and the participation of each driver and car in those accidents. Each driver in the system is uniquely identified by a driver ID, along with their name and address, and each car is uniquely identified by its registration number together with details such as model and manufacturing year. The system must allow storing ownership information that links a driver to one or more cars, while also allowing a car to be linked to one or more drivers if shared ownership occurs; duplicate ownership records for the same driver and car must not exist. Accident information must be stored using a unique report number assigned to each accident, along with the date on which the accident occurred and the location where it happened. Every accident reported in the system must have at least one participating driver and car, and this participation is recorded by linking the driver, the involved car, and the accident report together with the corresponding damage amount for that particular involvement. A participation record must reference an existing driver, an existing car, and an existing accident, and no two participation entries may repeat the same combination of driver, car, and accident report. The database must ensure that damage amounts are non-negative, accident dates are valid calendar dates, and car manufacturing years fall within reasonable limits. It must also preserve referential integrity so that ownership or participation entries cannot exist without valid driver, car, and accident information already present in the system. Deletion policies must prevent removal of drivers or cars that appear in past accident participation records unless historical consistency is preserved through controlled deletion rules or archival mechanisms. The system should maintain accurate links between drivers, cars, and accidents at all times, ensuring reliable retrieval of ownership histories, accident histories, and damage information for administrative, legal, and insurance-related purposes.

Entity Relationship Diagram



Schema Diagram



PERSON (driver_id: String, name: String, address: String)

CAR (reg_num: String, model: String, year: int)

ACCIDENT (report_num: int, accident_date: date, location: String)

OWNS (driver_id: String, reg_num: String)

PARTICIPATED (driver_id: String, reg_num: String, report_num: int, damage_amount: int)

Create Table

```
CREATE TABLE PERSON (
   driver_id VARCHAR (10) PRIMARY KEY,
   name VARCHAR (30),
   address VARCHAR (50)
);


CREATE TABLE CAR (
   reg_num VARCHAR (15) PRIMARY KEY,
```

```
    model VARCHAR (20),
    year INT,
    CONSTRAINT chk_year_range CHECK (year >= 1886 AND year <= 2100)
);


CREATE TABLE ACCIDENT (
    report_num INT PRIMARY KEY,
    accident_date DATE,
    location VARCHAR (50)
);

CREATE TABLE OWNS (
    driver_id VARCHAR (10),
    reg_num VARCHAR (15),
    PRIMARY KEY (driver_id, reg_num),
    FOREIGN KEY(driver_id) REFERENCES PERSON(driver_id)
        ON DELETE RESTRICT,
    FOREIGN KEY(reg_num) REFERENCES CAR(reg_num)
        ON DELETE RESTRICT
);

CREATE TABLE PARTICIPATED (
    driver_id VARCHAR (10),
    reg_num VARCHAR (15),
    report_num INT,
    damage_amount INT CHECK (damage_amount >= 0),
    PRIMARY KEY (driver_id, reg_num, report_num),
    FOREIGN KEY(driver_id) REFERENCES PERSON(driver_id)
        ON DELETE RESTRICT,
```

```
    FOREIGN KEY(reg_num) REFERENCES CAR(reg_num)
        ON DELETE RESTRICT,
    FOREIGN KEY(report_num) REFERENCES ACCIDENT(report_num)
        ON DELETE CASCADE
);
```

Structure of Table

desc person;

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
driver_id	varchar(10)	NO	PRI	NULL	
name	varchar(30)	YES		NULL	
address	varchar(50)	YES		NULL	

desc car;

Result Grid | Filter Rows: | Export: | Wra

Field	Type	Null	Key	Default	Extra
reg_num	varchar(15)	NO	PRI	NULL	
model	varchar(20)	YES		NULL	
year	int	YES		NULL	

desc accident;

Result Grid | Filter Rows: | Export: | Wrap

Field	Type	Null	Key	Default	Extra
report_num	int	NO	PRI	NULL	
accident_date	date	YES		NULL	
location	varchar(50)	YES		NULL	

desc owns;

Result Grid | Filter Rows: | Export: | W

Field	Type	Null	Key	Default	Extra
driver_id	varchar(10)	NO	PRI	NULL	
reg_num	varchar(15)	NO	PRI	NULL	

desc participated;

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
driver_id	varchar(10)	NO	PRI	NULL	
reg_num	varchar(15)	NO	PRI	NULL	
report_num	int	NO	PRI	NULL	
damage_amount	int	YES		NULL	

Inserting Values to Table

INSERT INTO PERSON VALUES

('A01','Richard','Srinivas nagar'),

('A02','Pradeep','Rajaji nagar'),

('A03','Smith','Ashok nagar'),

('A04','Venu','N R Colony'),

('A05','Jhon','Hanumanth nagar');

select * from person;

Result Grid | Filter Rows:

driver_id	name	address
A01	Richard	Srinivas nagar
A02	Pradeep	Rajaji nagar
A03	Smith	Ashok nagar
A04	Venu	N R Colony
A05	Jhon	Hanumanth nagar
NULL	NULL	NULL

INSERT INTO CAR VALUES

('KA052250','Indica',1990),

('KA031181','Lancer',1957),

('KA095477','Toyota',1998),

('KA053408','Honda',2008),

('KA041702','Audi',2005);

select * from car;

Result Grid | Filter Rows:

reg_num	model	year
KA031181	Lancer	1957
KA041702	Audi	2005
KA052250	Indica	1990
KA053408	Honda	2008
KA095477	Toyota	1998
NULL	NULL	NULL

INSERT INTO ACCIDENT VALUES

(11,'2003-01-01','Mysore Road'),

(12,'2004-02-02','South end Circle'),

(13,'2003-01-21','Bull temple Road'),

(14,'2008-02-17','Mysore Road'),

(15,'2005-03-04','Kanakpura Road');

select * from accident;

Result Grid | Filter Rows: | Edit:

report_num	accident_date	location
11	2003-01-01	Mysore Road
12	2004-02-02	South end Circle
13	2003-01-21	Bull temple Road
14	2008-02-17	Mysore Road
15	2005-03-04	Kanakpura Road
NULL	NULL	NULL

INSERT INTO OWNS VALUES

('A01','KA052250'),

('A02','KA053408'),

('A03','KA031181'),

('A04','KA095477'),

('A05','KA041702');

select * from owns;

Result Grid | Filter Rows:

driver_id	reg_num
A03	KA031181
A05	KA041702
A01	KA052250
A02	KA053408
A04	KA095477
NULL	NULL

INSERT INTO PARTICIPATED VALUES

('A01','KA052250',11,10000),

('A02','KA053408',12,50000),

('A03','KA095477',13,25000),

('A04','KA031181',14,3000),

('A05','KA041702',15,5000);

select * from participated;

Result Grid | Filter Rows: | Edit:

driver_id	reg_num	report_num	damage_amount
A01	KA052250	11	10000
A02	KA053408	12	50000
A03	KA095477	13	25000
A04	KA031181	14	3000
A05	KA041702	15	5000
NULL	NULL	NULL	NULL

Queries (Questions & Output)

iii. Display Accident date and location

SELECT accident_date, location

FROM ACCIDENT;

Result Grid | Filter Rows:

accident_date	location
2003-01-01	Mysore Road
2004-02-02	South end Circle
2003-01-21	Bull temple Road
2008-02-17	Mysore Road
2005-03-04	Kanakpura Road

iv. Update the damage amount to 25000 for the car with a specific reg_num (example 'K A053408') for which the accident report number was 12.

UPDATE PARTICIPATED

SET damage_amount = 25000

WHERE reg_num = 'KA053408'

AND report_num = 12;

select * from participated;

Result Grid | Filter Rows: | Edit:

driver_id	reg_num	report_num	damage_amount
A01	KA052250	11	10000
A02	KA053408	12	25000
A03	KA095477	13	25000
A04	KA031181	14	3000
A05	KA041702	15	5000
NULL	NULL	NULL	NULL

v. Add a new accident to the database.

INSERT INTO ACCIDENT (report_num, accident_date, location)

VALUES (16, '2024-03-10', 'MG Road');

select * from accident;

report_num	accident_date	location
11	2003-01-01	Mysore Road
12	2004-02-02	South end Circle
13	2003-01-21	Bull temple Road
14	2008-02-17	Mysore Road
15	2005-03-04	Kanakpura Road
16	2024-03-10	MG Road
NULL	NULL	NULL

vi. Display driver id who did accident with damage amount greater than or equal to Rs.25000

SELECT DISTINCT driver_id

FROM PARTICIPATED

WHERE damage_amount >= 25000;

Result Grid | Filte

driver_id
A02
A03

Experiment 2: More Queries on Insurance Database

Queries (Questions & Output)

i. Display the entire CAR relation in the ascending order of manufacturing year.

```
SELECT *

FROM CAR

ORDER BY year ASC;
```

reg_num	model	year
KA031181	Lancer	1957
KA052250	Indica	1990
KA095477	Toyota	1998
KA041702	Audi	2005
KA053408	Honda	2008
NULL	NULL	NULL

ii. Find the number of accidents in which cars belonging to a specific model (example 'Lancer') were involved.

```
SELECT COUNT (DISTINCT p. report_num)

FROM PARTICIPATED p

JOIN CAR c ON p.reg_num = c.reg_num

WHERE c. model = 'Lancer';
```

Result Grid | Filter Rows:

COUNT(DISTINCT p.report_num)
1

iii. Find the total number of people who owned cars that involved in accidents in 2008.

```
SELECT COUNT (DISTINCT o. driver_id)

FROM OWNS o

JOIN PARTICIPATED p ON o.reg_num = p.reg_num

JOIN ACCIDENT a ON p. report_num = a. report_num

WHERE YEAR (a. accident_date) = 2008;
```

Result Grid | Filter Rows:

COUNT(DISTINCT o.driver_id)
1

iv. List the entire participated relation in the Descending Order of Damage Amount.

```
SELECT *
FROM PARTICIPATED
ORDER BY damage_amount DESC;
```

Result Grid | Filter Rows: | Edit:

driver_id	reg_num	report_num	damage_amount
A02	KA053408	12	25000
A03	KA095477	13	25000
A01	KA052250	11	10000
A05	KA041702	15	5000
A04	KA031181	14	3000
NULL	NULL	NULL	NULL

v. Find the Average Damage Amount.

```
SELECT AVG(damage_amount) AS avg_damage
FROM PARTICIPATED;
```

Result Grid | Filter

avg_damage
13600.0000

vi. Delete the tuple whose Damage Amount is below the Average Damage Amount

```
DELETE FROM PARTICIPATED
WHERE damage_amount < (
   SELECT avg_damage
   FROM (
      SELECT AVG(damage_amount) AS avg_damage
      FROM PARTICIPATED
   ) AS temp
);
select * from participated;
SET SQL_SAFE_UPDATES = 1;
```

Result Grid | Filter Rows: | Edit:

driver_id	reg_num	report_num	damage_amount
A02	KA053408	12	25000
A03	KA095477	13	25000
NULL	NULL	NULL	NULL

vii. List the name of drivers whose Damage is Greater than the Average Damage Amount.

```
SELECT DISTINCT pe.name
FROM PERSON pe
JOIN PARTICIPATED p ON pe. driver_id = p. driver_id
WHERE p. damage_amount > (
   SELECT AVG(damage_amount)
   FROM PARTICIPATED
);
```

Result Grid | Filter Ro

	name
▶	Pradeep
	Smith

viii. Find Maximum Damage Amount.

```
SELECT MAX(damage_amount) AS max_damage
FROM PARTICIPATED;
```

Result Grid | Filter Rows:

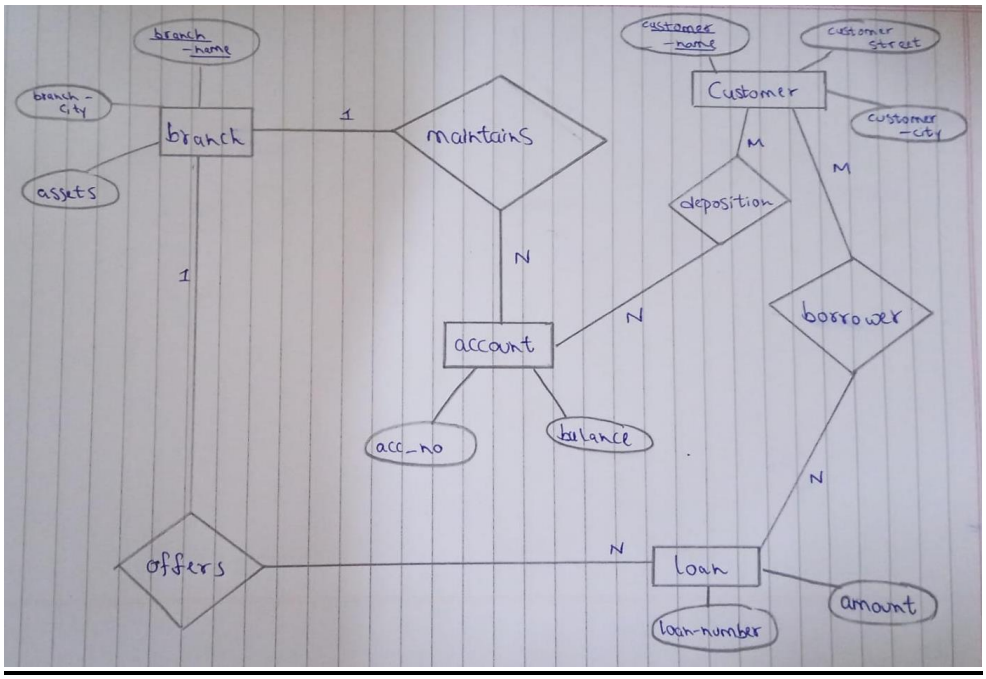
	max_damage
▶	25000

Experiment 3: Bank Database

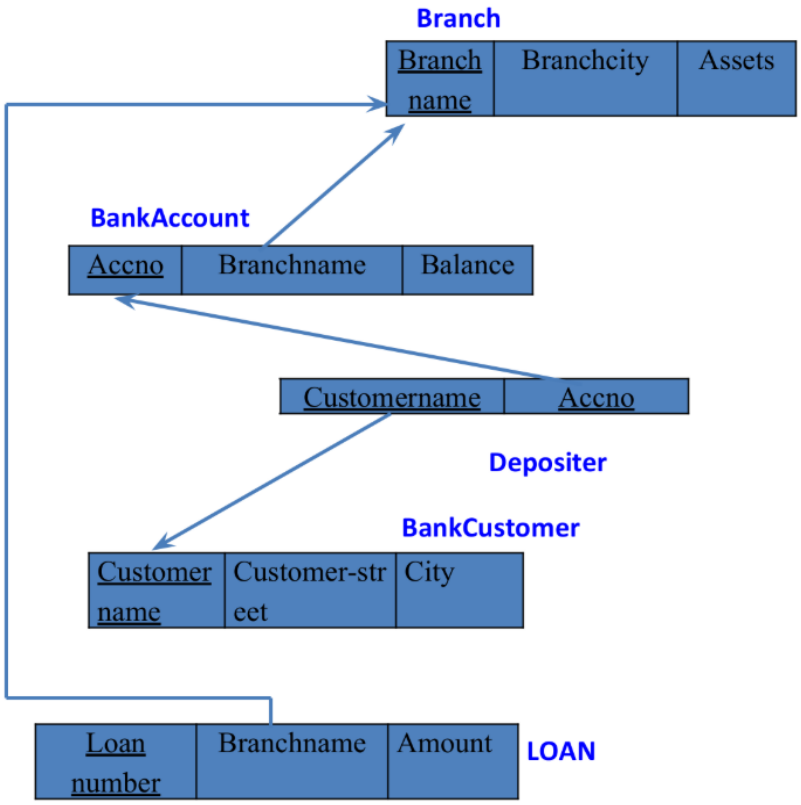
BANKING ENTERPRISE DATABASE

The banking system must store information about branches, bank accounts, customers, deposit relationships, and loans so that branch details (identified by branch name together with city and total assets) are linked to accounts and loans, each account (identified by an account number) records the branch it belongs to and the current balance, customers are recorded with their name, street and city, and a depositor relationship associates a customer with an account; loans are recorded by a unique loan number together with the branch name that issued the loan and the loan amount. Account numbers and loan numbers must be unique identifiers, branch names are used to associate accounts and loans to a branch, and customer names (as modelled) are used to identify customers referenced by depositor entries; every depositor entry must reference an existing customer and an existing account so that ownership and access relationships are always valid, and duplicate depositor records linking the same customer and account are disallowed. The system must maintain referential integrity so accounts cannot reference a non-existent branch, depositor rows cannot reference missing customers or accounts, and loans must reference an existing branch; deletion of a branch, account, or customer that is referenced by dependent records should be controlled (either disallowed or handled by archival/controlled reassignment) to preserve historical transaction and loan consistency. Numeric and temporal constraints must be enforced: account balances should be constrained to valid values (for example non-negative where overdraft is not allowed), branch assets and loan amounts must be non-negative and within specified business limits, and updates to balance or loan amounts should be auditable. Cardinality rules implied by the schema are enforced: a branch may host many account sand issue many loans, an account belongs to exactly one branch, a customer may be linked to many accounts through depositor relationships, and an account may have many depositors if joint accounts are permitted by policy. Implementation must prevent orphaned records, ensure uniqueness where required, and rely on application logic or database-level triggers to enforce complex rules such as cascading effects on deletion, business rules about allowed balance operations or overdrafts, and any required validation when transferring accounts between branches or when converting a customer's identifying details; the database should thus reliably support queries for branch-wise account lists, customer account ownership, account balances, and loan portfolios while preserving historical and referential integrity for auditing and regulatory reporting.

Entity & Relationship Diagram



Schema Diagram



Branch (branch-name: String, branch-city: String, assets: real)

BankAccount (accno: int, branch-name: String, balance: real)

BankCustomer (customer-name: String, customer-street: String, customer-city: String)

Depositer (customer-name: String, accno: int)

LOAN (loan-number: int, branch-name: String, amount: real)

Creating Tables

```
CREATE TABLE Branch (
   branch_name VARCHAR (30) PRIMARY KEY,
   branch_city VARCHAR (30) NOT NULL,
   assets DECIMAL (15,2) NOT NULL,
   CHECK (assets >= 0)
);


CREATE TABLE BankAccount (
   accno INT PRIMARY KEY,
   branch_name VARCHAR (30) NOT NULL,
   balance DECIMAL (15,2) NOT NULL,
   CHECK (balance >= 0),
   FOREIGN KEY (branch_name)
      REFERENCES Branch(branch_name)
      ON DELETE RESTRICT
      ON UPDATE CASCADE
);


CREATE TABLE BankCustomer (
```

```
    customer_name VARCHAR (30) PRIMARY KEY,
    customer_street VARCHAR (50),
    customer_city VARCHAR (30)
);


CREATE TABLE Depositer (
    customer_name VARCHAR (30),
    accno INT,
    PRIMARY KEY (customer_name, accno),
    FOREIGN KEY (customer_name)
        REFERENCES BankCustomer(customer_name)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (accno)
        REFERENCES BankAccount(accno)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE Loan (
    loan_number INT PRIMARY KEY,
    branch_name VARCHAR (30) NOT NULL,
    amount DECIMAL (15,2) NOT NULL,
    CHECK (amount >= 0),
    FOREIGN KEY (branch_name)
        REFERENCES Branch(branch_name)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
```

);

Structure of Tables

desc Branch;

Result Grid | Filter Rows: | Export: | Wrap Cel

Field	Type	Null	Key	Default	Extra
branch_name	varchar(30)	NO	PRI	NULL	
branch_city	varchar(30)	NO		NULL	
assets	decimal(15,2)	NO		NULL	

desc BankAccount;

Result Grid | Filter Rows: | Export: | Wrap Ce

Field	Type	Null	Key	Default	Extra
accno	int	NO	PRI	NULL	
branch_name	varchar(30)	NO	MUL	NULL	
balance	decimal(15,2)	NO		NULL	

desc BankCustomer;

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
customer_name	varchar(30)	NO	PRI	NULL	
customer_street	varchar(50)	YES		NULL	
customer_city	varchar(30)	YES		NULL	

desc depositer;

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
customer_name	varchar(30)	NO	PRI	NULL	
accno	int	NO	PRI	NULL	

desc Loan;

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
loan_number	int	NO	PRI	NULL	
branch_name	varchar(30)	NO	MUL	NULL	
amount	decimal(15,2)	NO		NULL	

Inserting Values into the Table

INSERT INTO Branch VALUES

('SBI_Chamrajpet','Bangalore',50000),

('SBI_ResidencyRoad','Bangalore',10000),

('SBI_ShivajiRoad','Bombay',20000),

('SBI_ParliamentRoad','Delhi',10000),

('SBI_Jantarmantar','Delhi',20000);

select *from Branch;

Result Grid | Filter Rows: | Edit:

branch_name	branch_city	assets
SBI_Chamrajpet	Bangalore	50000.00
SBI_Jantarmantar	Delhi	20000.00
SBI_ParliamentRoad	Delhi	10000.00
SBI_ResidencyRoad	Bangalore	10000.00
SBI_ShivajiRoad	Bombay	20000.00
NULL	NULL	NULL

INSERT INTO BankAccount VALUES

(1,'SBI_Chamrajpet',2000),

(2,'SBI_ResidencyRoad',5000),

(3,'SBI_ShivajiRoad',6000),

(4,'SBI_ParliamentRoad',9000),

(5,'SBI_Jantarmantar',8000);

select *from BankAccount;

Result Grid | Filter Rows:

accno	branch_name	balance
1	SBI_Chamrajpet	2000.00
2	SBI_ResidencyRoad	5000.00
3	SBI_ShivajiRoad	6000.00
4	SBI_ParliamentRoad	9000.00
5	SBI_Jantarmantar	8000.00
NULL	NULL	NULL

INSERT INTO BankCustomer VALUES

('Avinash','Bull_Temple_Road','Bangalore'),

('Dinesh','Bannerghatta_Road','Bangalore'),

('Mohan','NationalCollege_Road','Bangalore'),

('Nikil','Akbar_Road','Delhi'),

('Ravi','Prithviraj_Road','Delhi');

select *from BankCustomer;

customer_name	customer_street	customer_city
Avinash	Bull_Temple_Road	Bangalore
Dinesh	Bannerghatta_Road	Bangalore
Mohan	NationalCollege_Road	Bangalore
Nikil	Akbar_Road	Delhi
Ravi	Prithviraj_Road	Delhi
NULL	NULL	NULL

INSERT INTO Depositer VALUES

('Avinash',1),

('Dinesh',2),

('Nikil',4),

('Ravi',5),

('Avinash',3);

select *from depositer;

Result Grid | Filter Rows:

customer_name	accno
Avinash	1
Dinesh	2
Avinash	3
Nikil	4
Ravi	5
NULL	NULL

INSERT INTO Loan VALUES

(1,'SBI_Chamrajpet',1000),

(2,'SBI_ResidencyRoad',2000),

(3,'SBI_ShivajiRoad',3000),

(4,'SBI_ParliamentRoad',4000),

(5,'SBI_Jantarmantar',5000);

select *from Loan;

Result Grid | Filter Rows: | Ed

loan_number	branch_name	amount
1	SBI_Chamrajpet	1000.00
2	SBI_ResidencyRoad	2000.00
3	SBI_ShivajiRoad	3000.00
4	SBI_ParliamentRoad	4000.00
5	SBI_Jantarmantar	5000.00
NULL	NULL	NULL

Queries (Questions & Output)

iii. Display the branch name and assets from all branches in lakhs of rupees and rename the assets column to 'assets in lakhs'.

SELECT branch_name, assets/100000 AS `assets in lakhs`

FROM Branch;

Result Grid | Filter Rows:

branch_name	assets in lakhs
SBI_Chamrajpet	0.500000
SBI_Jantarmantar	0.200000
SBI_ParliamentRoad	0.100000
SBI_ResidencyRoad	0.100000
SBI_ShivajiRoad	0.200000

iv. Find all the customers who have at least two accounts at the same branch (ex. SBI_ResidencyRoad).

SELECT d. customer_name

FROM Depositer d

JOIN BankAccount b ON d. accno = b. accno

WHERE b. branch_name = 'SBI_ResidencyRoad'

GROUP BY d. customer_name

HAVING COUNT (*) >= 2;

Result Grid | Filter Row

customer_name

v. Create a view which gives each branch the sum of the amount of all the loans at the branch.

CREATE VIEW BranchLoanSum AS

SELECT branch_name, SUM(amount) AS total_loan

FROM Loan

GROUP BY branch_name;

Result Grid | Filter Rows:

branch_name	total_loan
SBI_Chamrajpet	1000.00
SBI_Jantarmantar	5000.00
SBI_ParliamentRoad	4000.00
SBI_ResidencyRoad	2000.00
SBI_ShivajiRoad	3000.00

Experiment 4: More Queries on Bank Database

Queries (Questions & Output)

i. Find all the customers who have an account at all the branches located in a specific city (Ex. Delhi).

```
SELECT c. customer_name
FROM BankCustomer c
JOIN Depositer d ON c. customer_name = d. customer_name
JOIN BankAccount b ON d. accno = b. accno
JOIN Branch br ON b. branch_name = br. branch_name
WHERE br. branch_city = 'Delhi'
GROUP BY c. customer_name
HAVING COUNT (DISTINCT br. branch_name) = (
   SELECT COUNT (*)
   FROM Branch
   WHERE branch_city = 'Delhi'
);
```

Result Grid | Filter Rows:

	customer_name

ii. Find all customers who have a loan at the bank but do not have an account.

```
SELECT DISTINCT l. loan_number
FROM Loan l
LEFT JOIN BankAccount b ON l. branch_name = b. branch_name
WHERE b. accno IS NULL;
```

Result Grid | Filter Rows

	loan_number
▶	3

iii. Find all customers who have both an account and a loan at the Bangalore branch

```
SELECT DISTINCT d. customer_name
FROM Depositer d
JOIN BankAccount b ON d. accno = b. accno
```

```
JOIN Loan l ON b. branch_name = l. branch_name

JOIN Branch br ON b. branch_name = br. branch_name

WHERE br. branch_city = 'Bangalore';
```

Result Grid | Filter Rows:

customer_name
Avinash
Dinesh

iv. Find the names of all branches that have greater assets than all branches located in Bangalore.

```
SELECT branch_name

FROM Branch

WHERE assets > ALL (

   SELECT assets

   FROM Branch

   WHERE branch_city = 'Bangalore'

);
```

Result Grid | Filter Rows:

branch_name
NULL

v. Demonstrate how you delete all account tuples at every branch located in a specific city (Ex. Bombay).

```
SET SQL_SAFE_UPDATES=0;

DELETE FROM Depositer

WHERE accno IN (

   SELECT accno

   FROM BankAccount

   WHERE branch_name IN (

      SELECT branch_name

      FROM Branch

      WHERE branch_city = 'Bombay'

   )

);
```

```
DELETE FROM BankAccount
WHERE branch_name IN (
   SELECT branch_name
   FROM Branch
   WHERE branch_city = 'Bombay'
);
SELECT * FROM BankAccount;
SET SQL_SAFE_UPDATES=1;
```

Result Grid | Filter Rows:

accno	branch_name	balance
1	SBI_Chamrajpet	2000.00
2	SBI_ResidencyRoad	5000.00
4	SBI_ParliamentRoad	9000.00
5	SBI_Jantarmantar	8000.00
NULL	NULL	NULL

vi. Update the Balance of all accounts by 5%

```
UPDATE BankAccount
SET balance = balance * 1.05;
SELECT * FROM BankAccount;
```

Result Grid | Filter Rows:

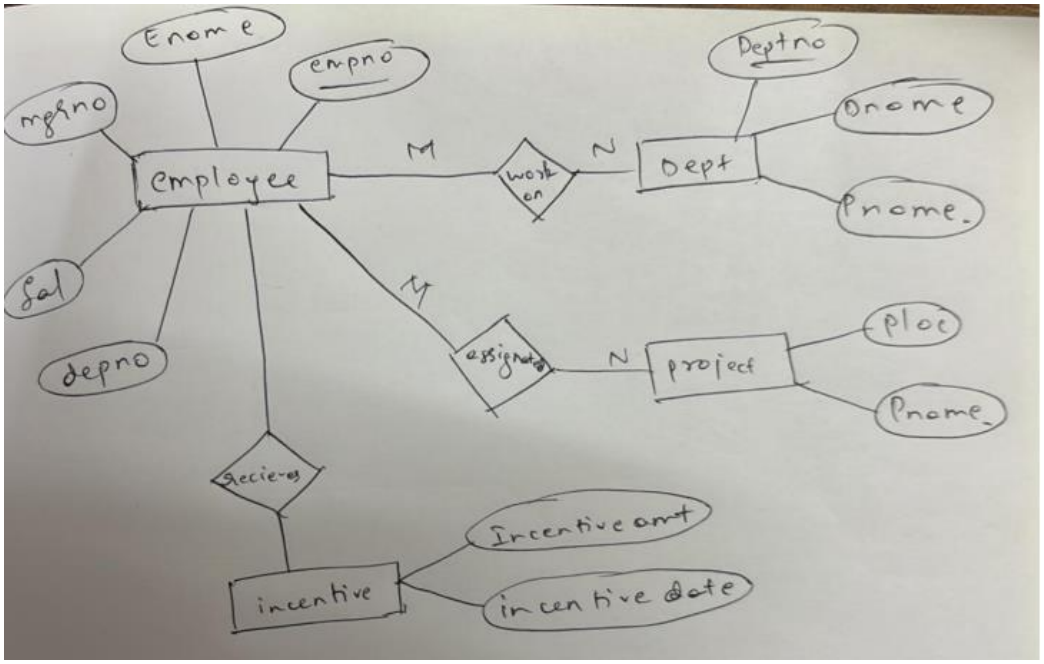
accno	branch_name	balance
1	SBI_Chamrajpet	2100.00
2	SBI_ResidencyRoad	5250.00
4	SBI_ParliamentRoad	9450.00
5	SBI_Jantarmantar	8400.00
NULL	NULL	NULL

Experiment 5 – Employee Database

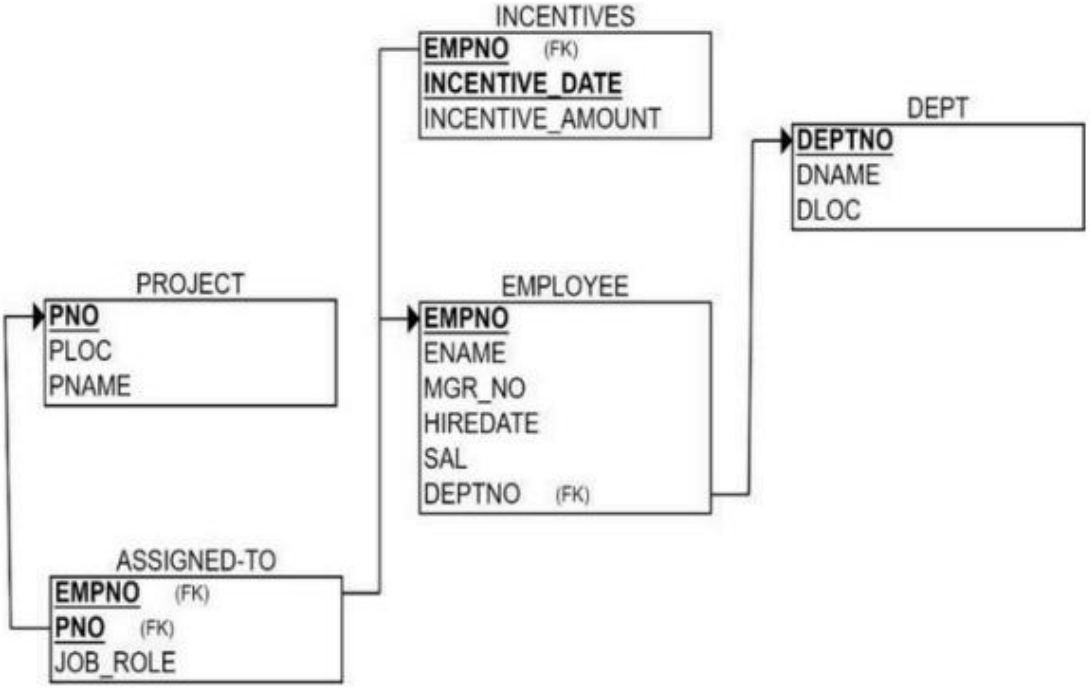
Employee Database

The employee database must record each employee's identifying number, name, manager reference, hire date, salary, and department affiliation while also tracking departmental details, project assignments (including the role an employee plays on a project), and any incentive payments given to employees. Every employee is represented by a unique employee number and has a hire date and salary that must be valid; the manager field is a self-referencing link that must, if present, point to an existing employee and must never create a circular management chain or reference the employee themself. Departments are identified by a unique department number and include a department name and location; every department referenced by an employee or by other structures must exist in the department table, and departments may contain zero or many employees. Projects are recorded with a unique project number, project name and project location; employees may be assigned to multiple projects and each project may have many employees, with each assignment carrying the employee's job role for that project — duplicate assignments of the same employee to the same project are disallowed. Incentive payments are recorded with the employee reference, the incentive date and the incentive amount; an incentive entry must reference an existing employee and incentive amounts must be non-negative and dated on or after the employee's hire date. Referential integrity must be enforced so that employee records cannot reference non-existent departments, projects, or managers, and assignment and incentive records cannot exist without corresponding employee, project, or department records as appropriate. Salary, incentive amounts, and any monetary fields must be constrained to valid numeric ranges and hire/ incentive dates must be valid calendar dates (and typically not future-dated unless business rules permit). Deletion and update policies must preserve historical consistency: deleting an employee who appears as a manager, as a project assignee, or in incentive records should be prevented or should be handled via controlled archival, reassignment, or soft-delete flags rather than hard deletion to preserve audit trails; similarly, changing a department or project identifier must either be disallowed if it would orphan historical records or handled by introducing immutable surrogate keys. Business rules include preventing circular manager chains, ensuring an employee's manager (if specified) cannot be the employee themself, disallowing duplicate project-assignments, requiring that incentive dates fall within the employee's employment window, and optionally requiring at least one project assignment or at least one incentive record depending on policy for reporting. Implementation should use primary-key and foreign-key constraints for identity and linkage, unique constraints to prevent duplicate assignments, check constraints for monetary and date ranges, and application logic or triggers for complex temporal or graph constraints (like cycle detection in management relationships and enforcing non-overlap or other schedule-related rules if assignments gain temporal attributes later). The system must therefore reliably support queries such as employee reporting lines, department staffing lists, project rosters with job roles, incentive payment histories, salary analyses, and audit reports while maintaining data integrity, preventing inconsistent deletions, and preserving a complete historical record for HR and compliance needs.

Entity & Relashionship Diagram



Schema Diagram



DEPT(DEPTNO PK)

EMPLOYEE(EMPNO PK, DEPTNO FK, MGR_NO FK)

PROJECT(PNO PK)

ASSIGNED_TO(EMPNO FK, PNO FK, Composite PK)

INCENTIVES(EMPNO FK, INCENTIVE_DATE PK)

Creating Tables

```
CREATE TABLE DEPT (
   DEPTNO INT PRIMARY KEY,
   DNAME VARCHAR(20),
   DLOC VARCHAR(20)
);
CREATE TABLE EMPLOYEE (
   EMPNO INT PRIMARY KEY,
   ENAME VARCHAR(20),
   MGR_NO INT,
   HIREDATE DATE,
   SAL INT,
   DEPTNO INT,
   FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO),
   FOREIGN KEY (MGR_NO) REFERENCES EMPLOYEE(EMPNO)
);
CREATE TABLE PROJECT (
   PNO INT PRIMARY KEY,
   PNAME VARCHAR(20),
   PLOC VARCHAR(20)
);
```

```
CREATE TABLE ASSIGNED_TO (
    EMPNO INT,
    PNO INT,
    JOB_ROLE VARCHAR(20),
    PRIMARY KEY (EMPNO, PNO),
    FOREIGN KEY (EMPNO) REFERENCES EMPLOYEE(EMPNO),
    FOREIGN KEY (PNO) REFERENCES PROJECT(PNO)
);
CREATE TABLE INCENTIVES (
    EMPNO INT,
    INCENTIVE_DATE DATE,
    INCENTIVE_AMOUNT INT,
    PRIMARY KEY (EMPNO, INCENTIVE_DATE),
    FOREIGN KEY (EMPNO) REFERENCES EMPLOYEE(EMPNO)
);
```

Structure of Tables

desc dept;

Field	Type	Null	Key	Default	Extra
DEPTNO	int	NO	PRI	NULL	
DNAME	varchar(20)	YES		NULL	
DLOC	varchar(20)	YES		NULL	

desc employee;

Field	Type	Null	Key	Default	Extra
EMPNO	int	NO	PRI	NULL	
ENAME	varchar(20)	YES		NULL	
MGR_NO	int	YES	MUL	NULL	
HIREDATE	date	YES		NULL	
SAL	int	YES		NULL	

desc project;

Field	Type	Null	Key	Default	Extra
PNO	int	NO	PRI	NULL	
PNAME	varchar(20)	YES		NULL	
PLOC	varchar(20)	YES		NULL	

desc assigned_to;

Field	Type	Null	Key	Default	Extra
EMPNO	int	NO	PRI	NULL	
PNO	int	NO	PRI	NULL	
JOB_ROLE	varchar(20)	YES		NULL	

desc incentives;

Field	Type	Null	Key	Default	Extra
EMPNO	int	NO	PRI	NULL	
INCENTIVE_DATE	date	NO	PRI	NULL	
INCENTIVE_AMOUNT	int	YES		NULL	

Inserting Values into the Table

INSERT INTO DEPT VALUES

(10,'HR','Bengaluru'),

(20,'IT','Hyderabad'),

(30,'Finance','Mysuru'),

(40,'Admin','Bengaluru'),

(50,'Sales','Hyderabad');

select * from dept;

DEPTNO	DNAME	DLOC
10	HR	Bengaluru
20	IT	Hyderabad
30	Finance	Mysuru
40	Admin	Bengaluru
50	Sales	Hyderabad

INSERT INTO EMPLOYEE VALUES

(101,'Asha',NULL,'2015-01-10',90000,10),

(102,'Ravi',101,'2016-03-15',60000,10),

(103,'Kiran',101,'2017-06-20',55000,10),

(104,'Meena',NULL,'2014-04-12',95000,20),

(105,'Suresh',104,'2018-08-01',65000,20),

(106,'Neha',104,'2019-05-18',60000,20),

(107,'Rahul',NULL,'2013-09-23',98000,30),

(108,'Anita',107,'2018-01-11',62000,30);

select * from employee;

EMPNO	ENAME	MGR_NO	HIREDATE	SAL	DEPTNO
101	Asha	NULL	2015-01-10	90000	10
102	Ravi	101	2016-03-15	60000	10
103	Kiran	101	2017-06-20	55000	10
104	Meena	NULL	2014-04-12	95000	20
105	Suresh	104	2018-08-01	65000	20

INSERT INTO PROJECT VALUES

(1,'Payroll','Bengaluru'),

(2,'ERP','Hyderabad'),

(3,'Audit','Mysuru'),

(4,'CRM','Bengaluru'),

(5,'SalesApp','Hyderabad');

select * from project;

PNO	PNAME	PLOC
1	Payroll	Bengaluru
2	ERP	Hyderabad
3	Audit	Mysuru
4	CRM	Bengaluru
5	SalesApp	Hyderabad

INSERT INTO ASSIGNED_TO VALUES

(102,1,'Developer'),

(103,1,'Tester'),

(105,2,'Developer'),

(106,2,'Tester'),

(108,3,'Analyst'),

(102,4,'Support'),

(105,5,'Consultant');

select * from assigned_to;

	EMPNO	PNO	JOB_ROLE
▶	102	1	Developer
	102	4	Support
	103	1	Tester
	105	2	Developer
	105	5	Consultant

INSERT INTO INCENTIVES VALUES

(102,'2019-01-10',5000),

(103,'2019-01-15',7000),

(105,'2019-01-20',6000),

(106,'2019-02-10',8000),

(108,'2019-01-25',9000),

(105,'2019-03-01',4000);

select * from incentives;

	EMPNO	INCENTIVE_DATE	INCENTIVE_AMOUNT
▶	102	2019-01-10	5000
	103	2019-01-15	7000
	105	2019-01-20	6000
	105	2019-03-01	4000
	106	2019-02-10	8000

Queries (Questions & Output)

iii. Retrieve the employee numbers of all employees who work on project located in Bengaluru, Hyderabad, or Mysuru

```
SELECT DISTINCT A.EMPNO

FROM ASSIGNED_TO A

JOIN PROJECT P ON A.PNO = P.PNO

WHERE P.PLOC IN ('Bengaluru','Hyderabad','Mysuru');
```

	EMPNO
▶	102
	103
	105
	106
	108

iv. Get Employee ID's of those employees who didn't receive incentives

```
SELECT EMPNO

FROM EMPLOYEE

WHERE EMPNO NOT IN (SELECT EMPNO FROM INCENTIVES);
```

	EMPNO
▶	101
	104
	107

v. Write a SQL query to find the employees name, number, dept, job_role, department location and project location who are working for a project location same as his/her department location.

```
SELECT E.ENAME, E.EMPNO, D.DNAME, A.JOB_ROLE,

    D.DLOC AS DEPT_LOCATION, P.PLOC AS PROJECT_LOCATION

FROM EMPLOYEE E

JOIN DEPT D ON E.DEPTNO = D.DEPTNO

JOIN ASSIGNED_TO A ON E.EMPNO = A.EMPNO

JOIN PROJECT P ON A.PNO = P.PNO
```

WHERE D.DLOC = P.PLOC;

ENAME	EMPNO	DNAME	JOB_ROLE	DEPT_LOCATION	PROJECT_LOCATION
Ravi	102	HR	Developer	Bengaluru	Bengaluru
Kiran	103	HR	Tester	Bengaluru	Bengaluru
Suresh	105	IT	Developer	Hyderabad	Hyderabad
Neha	106	IT	Tester	Hyderabad	Hyderabad
Anita	108	Finance	Analyst	Mysuru	Mysuru

Experiment 6: More Queries on Employee Database

Queries (Questions & Output)

iii. List the name of the managers with the maximum employees

```
SELECT M.ENAME
FROM EMPLOYEE E
JOIN EMPLOYEE M ON E.MGR_NO = M.EMPNO
GROUP BY M.EMPNO, M.ENAME
HAVING COUNT(*) = (
  SELECT MAX(CNT)
  FROM (
    SELECT COUNT(*) AS CNT
    FROM EMPLOYEE
    WHERE MGR_NO IS NOT NULL
    GROUP BY MGR_NO
  ) AS T
);
```

ENAME
Asha
Meena

iv. Display those managers name whose salary is more than average salary of his employee.

SELECT M.ENAME

```
FROM EMPLOYEE M
WHERE M.SAL >
(
   SELECT AVG(E.SAL)
   FROM EMPLOYEE E
   WHERE E.MGR_NO = M.EMPNO
);
```

	ENAME
▶	Asha
	Meena
	Rahul

v. Find the name of the second top level managers of each department.

```
SELECT ENAME
FROM (
   SELECT ENAME, DEPTNO,
         DENSE_RANK() OVER (PARTITION BY DEPTNO ORDER BY SAL DESC) AS DR
   FROM EMPLOYEE
) AS T
WHERE DR = 2;
```

	ENAME
▶	Ravi
	Suresh
	Anita

vi. Find the employee details who got second maximum incentive in January 2019.

```
SELECT *
FROM INCENTIVES
WHERE INCENTIVE_AMOUNT = (
   SELECT MAX(INCENTIVE_AMOUNT)
   FROM INCENTIVES
   WHERE INCENTIVE_DATE BETWEEN '2019-01-01' AND '2019-01-31'
   AND INCENTIVE_AMOUNT <
      (SELECT MAX(INCENTIVE_AMOUNT)
```

```
    FROM INCENTIVES
    WHERE INCENTIVE_DATE BETWEEN '2019-01-01' AND '2019-01-31')
);
```

	EMPNO	INCENTIVE_DATE	INCENTIVE_AMOUNT
▶	103	2019-01-15	7000

vii. Display those employees who are working in the same department where his manager is working.

```
SELECT E.*
FROM EMPLOYEE E
JOIN EMPLOYEE M ON E.MGR_NO = M.EMPNO
WHERE E.DEPTNO = M.DEPTNO;
```

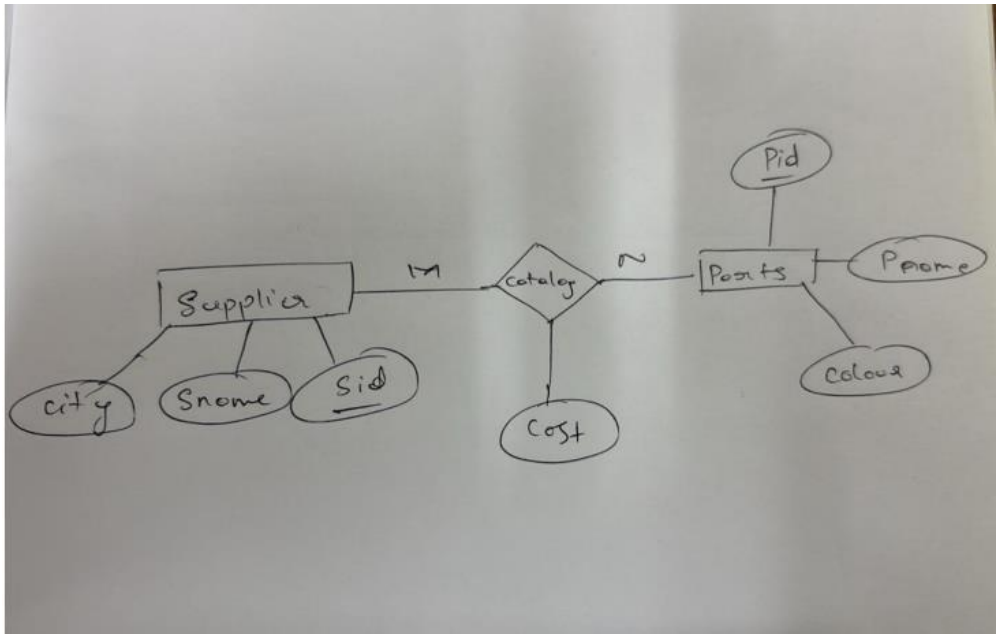
	EMPNO	ENAME	MGR_NO	HIREDATE	SAL	DEPTNO
▶	102	Ravi	101	2016-03-15	60000	10
	103	Kiran	101	2017-06-20	55000	10
	105	Suresh	104	2018-08-01	65000	20
	106	Neha	104	2019-05-18	60000	20
	108	Anita	107	2018-01-11	62000	30

Experiment 7 – Supplier Database

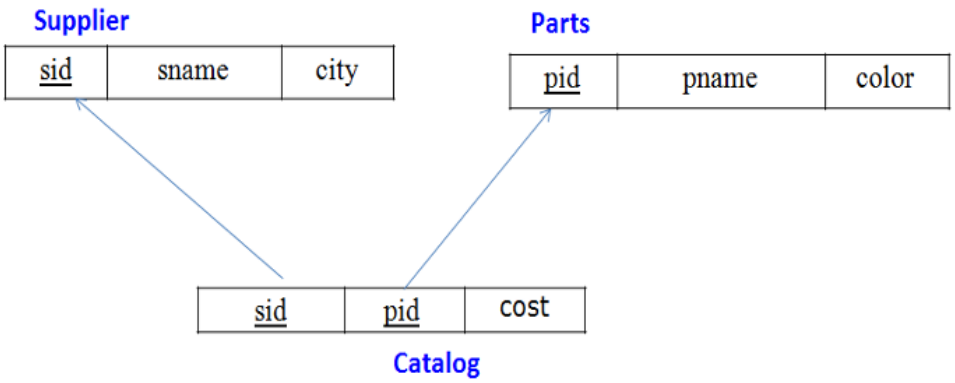
Supplier Database

The supplier database must store information about suppliers, the parts they provide, and the prices at which each part is offered so that purchasing, analysis, and reporting can be done accurately. Each supplier is uniquely identified by a supplier ID and is recorded with a name and the city in which the supplier is located; each part is uniquely identified by a part ID and includes a part name and a color. The system must maintain a catalog that links suppliers to the parts they supply and records the cost at which a given supplier sells a given part. Every catalog entry must reference an existing supplier and an existing part, and there must be no duplicate entries for the same combination of supplier and part, so that at most one current price record exists per supplier–part pair. Costs must be valid numeric values and strictly non-negative, and business rules may specify upper limits or currency formats that must be enforced consistently. The data model must support the possibility that a supplier can provide many different parts, that a part can be supplied by many different suppliers, and that some suppliers or parts may temporarily have no catalog entries if they are inactive or not currently traded. Referential integrity must be enforced so that a supplier or part cannot be deleted while still referenced in the catalog unless such deletion is handled by controlled archival or cascade rules that preserve historical price information; in general, historical catalog data should not be lost, as it may be required for audits or trend analysis. The system should allow queries such as “find all suppliers for a given part,” “list all parts provided by a given supplier,” “retrieve the cheapest supplier for each part,” and “analyze supplier coverage by city,” and must therefore guarantee that identifiers are unique, relationships between suppliers, parts, and catalog entries are consistent, and price information is accurate and reliably maintained over time.

Entity & Relashionship Diagram



Schema Diagram



Supplier (sid, sname, city)

Parts (pid, pname, color)

Catalog (sid, pid, cost)

Creating Tables

```
CREATE TABLE Supplier (
   sid INT PRIMARY KEY,
   sname VARCHAR(30),
   city VARCHAR(30)
);

CREATE TABLE Parts (
   pid INT PRIMARY KEY,
   pname VARCHAR(30),
   color VARCHAR(20)
);

CREATE TABLE Catalog (
```

```
    sid INT,
    pid INT,
    cost DECIMAL(10,2),
    PRIMARY KEY (sid, pid),
    FOREIGN KEY (sid) REFERENCES Supplier(sid),
    FOREIGN KEY (pid) REFERENCES Parts(pid)
);
```

Structure of Tables

desc supplier;

Field	Type	Null	Key	Default	Extra
sid	int	NO	PRI	NULL	
sname	varchar(30)	YES		NULL	
city	varchar(30)	YES		NULL	

desc parts;

Field	Type	Null	Key	Default	Extra
pid	int	NO	PRI	NULL	
pname	varchar(30)	YES		NULL	
color	varchar(20)	YES		NULL	

desc catalog;

Field	Type	Null	Key	Default	Extra
sid	int	NO	PRI	NULL	
pid	int	NO	PRI	NULL	
cost	decimal(10,2)	YES		NULL	

Inserting Values into Table

INSERT INTO Supplier VALUES

(1, 'Acme Widget Suppliers', 'Bangalore'),

(2, 'Global Parts Ltd', 'Mumbai'),

(3, 'Universal Supplies', 'Delhi');

Select * from supplier;

sid	sname	city
1	Acme Widget Suppliers	Bangalore
2	Global Parts Ltd	Mumbai
3	Universal Supplies	Delhi
NULL	NULL	NULL

INSERT INTO Parts VALUES

(101, 'Bolt', 'Red'),

(102, 'Nut', 'Green'),

(103, 'Screw', 'Red'),

(104, 'Washer', 'Blue');

Select * from parts;

pid	pname	color
101	Bolt	Red
102	Nut	Green
103	Screw	Red
104	Washer	Blue
NULL	NULL	NULL

INSERT INTO Catalog VALUES

(1, 101, 50),

(1, 102, 40),

(1, 103, 60),

(2, 101, 55),

(2, 104, 45),

(3, 102, 42),

(3, 103, 65),

(3, 104, 50);

Select * from catalog;

sid	pid	cost
1	101	50.00
1	102	40.00
1	103	60.00
2	101	55.00
2	104	45.00
3	102	42.00
3	103	65.00
3	104	50.00
NULL	NULL	NULL

Queries(Questions & Answers)

iii. Find the pnames of parts for which there is some supplier.

```
SELECT DISTINCT p.pname
FROM Parts p
JOIN Catalog c ON p.pid = c.pid;
```

pname
Bolt
Nut
Screw
Washer

iv. Find the snames of suppliers who supply every part.

```
SELECT s.sname
FROM Supplier s
WHERE NOT EXISTS (
   SELECT *
   FROM Parts p
   WHERE NOT EXISTS (
      SELECT *
      FROM Catalog c
```

```
        WHERE c.sid = s.sid AND c.pid = p.pid
    )
);
```

	sname

v. Find the snames of suppliers who supply every red part.

```
SELECT s.sname
FROM Supplier s
WHERE NOT EXISTS (
    SELECT *
    FROM Parts p
    WHERE p.color = 'Red'
    AND NOT EXISTS (
        SELECT *
        FROM Catalog c
        WHERE c.sid = s.sid AND c.pid = p.pid
    )
);
```

	sname
▶	Acme Widget Suppliers

vi. Find the pnames of parts supplied by Acme Widget Suppliers and by no one else.

```
SELECT p.pname
FROM Parts p
JOIN Catalog c ON p.pid = c.pid
```

```
JOIN Supplier s ON s.sid = c.sid
WHERE s.sname = 'Acme Widget Suppliers'
AND p.pid NOT IN (
   SELECT pid
   FROM Catalog c2
   JOIN Supplier s2 ON c2.sid = s2.sid
   WHERE s2.sname <> 'Acme Widget Suppliers'
);
```

	pname

vii. Find the sids of suppliers who charge more for some part than the average cost of that part (averaged over all the suppliers who supply that part).

```
SELECT DISTINCT c1.sid
FROM Catalog c1
WHERE c1.cost > (
   SELECT AVG(c2.cost)
   FROM Catalog c2
   WHERE c2.pid = c1.pid
);
```

	sid
▶	2
	3

viii. For each part, find the sname of the supplier who charges the most for that part.

```
SELECT p.pname, s.sname
```

```
FROM Parts p
JOIN Catalog c ON p.pid = c.pid
JOIN Supplier s ON s.sid = c.sid
WHERE c.cost = (
    SELECT MAX(c2.cost)
    FROM Catalog c2
    WHERE c2.pid = p.pid
);
```

pname	sname
Bolt	Global Parts Ltd
Nut	Universal Supplies
Screw	Universal Supplies
Washer	Universal Supplies

Experiment 8: More Queries on Supplier Database

1. Find the most expensive part overall and the supplier who supplies it.

```
SELECT p.pname, s.sname, c.cost
FROM Catalog c
JOIN Parts p ON c.pid = p.pid
JOIN Supplier s ON c.sid = s.sid
WHERE c.cost = (
   SELECT MAX(cost) FROM Catalog
);
```

	pname	sname	cost
▶	Screw	Universal Supplies	65.00

2. Find suppliers who do NOT supply any red parts.

```
SELECT s.sname
FROM Supplier s
WHERE s.sid NOT IN (
   SELECT c.sid
   FROM Catalog c
   JOIN Parts p ON c.pid = p.pid
   WHERE p.color = 'Red'
);
```

	sname

3. Show each supplier and total value of all parts they supply.

```
SELECT s.sname, SUM(c.cost) AS total_value
FROM Supplier s
JOIN Catalog c ON s.sid = c.sid
GROUP BY s.sname;
```

	sname	total_value
▶	Acme Widget Suppliers	150.00
	Global Parts Ltd	100.00
	Universal Supplies	157.00

4. Find suppliers who supply at least 2 parts cheaper than ₹20.

```
SELECT s.sname
FROM Supplier s
JOIN Catalog c ON s.sid = c.sid
WHERE c.cost < 20
GROUP BY s.sname
HAVING COUNT(DISTINCT c.pid) >= 2;
```

	sname

5. List suppliers who offer the cheapest cost for each part.

```
SELECT p.pname, s.sname, c.cost
FROM Catalog c
JOIN Parts p ON c.pid = p.pid
JOIN Supplier s ON c.sid = s.sid
WHERE c.cost = (
   SELECT MIN(c2.cost)
   FROM Catalog c2
   WHERE c2.pid = c.pid
);
```

	pname	sname	cost
▶	Bolt	Acme Widget Suppliers	50.00
	Nut	Acme Widget Suppliers	40.00
	Screw	Acme Widget Suppliers	60.00
	Washer	Global Parts Ltd	45.00

6. Create a view showing suppliers and the total number of parts they supply.

```
CREATE VIEW Supplier_Part_Count AS
SELECT s.sname, COUNT(DISTINCT c.pid) AS total_parts
FROM Supplier s
JOIN Catalog c ON s.sid = c.sid
GROUP BY s.sname;
SELECT * FROM Supplier_Part_Count;
```

	sname	total_parts
▶	Acme Widget Suppliers	3
	Global Parts Ltd	2
	Universal Supplies	3

7. Create a view of the most expensive supplier for each part.

```
CREATE VIEW Most_Expensive_Supplier AS
SELECT p.pname, s.sname, c.cost
FROM Catalog c
JOIN Parts p ON c.pid = p.pid
JOIN Supplier s ON c.sid = s.sid
WHERE c.cost = (
   SELECT MAX(c2.cost)
   FROM Catalog c2
   WHERE c2.pid = c.pid
);
SELECT * FROM Most_Expensive_Supplier;
```

	pname	sname	cost
▶	Bolt	Global Parts Ltd	55.00
	Nut	Universal Supplies	42.00
	Screw	Universal Supplies	65.00
	Washer	Universal Supplies	50.00

8. Create a Trigger to prevent inserting a Catalog cost below 1.

```
DELIMITER $$

CREATE TRIGGER check_cost_before_insert
BEFORE INSERT ON Catalog
FOR EACH ROW
BEGIN
   IF NEW.cost < 1 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cost must be at least 1';
   END IF;
END$$

DELIMITER ;
```

9. Create a trigger to set to default cost if not provided.

```
DELIMITER $$

CREATE TRIGGER set_default_cost
BEFORE INSERT ON Catalog
FOR EACH ROW
BEGIN
   IF NEW.cost IS NULL THEN
      SET NEW.cost = 10;
   END IF;
END$$

DELIMITER ;
```

Experiment 5: NoSQL Cloud Installation (MongoDB Atlas)

Aim:

To install and configure a NoSQL database in the cloud using MongoDB Atlas and perform basic database operations.

Software Requirements:

- Web browser (Chrome / Firefox)
- Internet connection
- MongoDB Atlas account

Procedure:

1. Open a web browser and visit https://www.mongodb.com/atlas.
2. Click on **Sign Up** and create a MongoDB Atlas account using email or Google login.
3. After successful login, create a **new project**.
4. Click on **Build a Database** and choose the **Free Tier (M0)** cluster.
5. Select the cloud provider and region (preferably nearest region).
6. Create the cluster and wait for deployment to complete.
7. Configure **Database Access** by creating a database user with username and password.
8. Configure **Network Access** by allowing access from the current IP address (or 0.0.0.0/0).
9. Click **Connect** and obtain the MongoDB connection string.
10. Use MongoDB Compass or MongoDB Shell to connect to the cloud database using the provided connection string.
11. Create a database and collections and perform basic operations such as insert, find, update, and delete.

Result:

MongoDB Atlas cloud database was successfully created and connected. Basic NoSQL operations were performed on the cloud-hosted database.

Conclusion:

Thus, a NoSQL database was successfully installed and configured in the cloud using MongoDB Atlas, demonstrating cloud-based database deployment and access.

Experiment 8: NoSQL – Student Database

Specification of NoSQL – Student Database Application:

The NoSQL student database must store and manage student information using a document-oriented data model in MongoDB. Each student record is uniquely identified by a roll number and includes attributes such as age, contact number, and email ID. The database must support insertion of appropriate student records and allow modification of existing data, including updating a student's email ID based on roll number and replacing a student name with a new value for a specified roll number. It should also support administrative operations such as exporting the student collection to the local file system for backup or data transfer purposes and importing data from external CSV files into MongoDB collections. Additionally, the system must allow safe deletion of collections when required. The database should ensure flexible schema handling, efficient document updates, and reliable data persistence while demonstrating fundamental NoSQL operations such as insert, update, replace, export, import, and drop.

Create Database:

```
> use Student;
< switched to db Student
```

Insert Appropriate Values:

Insert at least 4 documents each with RollNo, Name, Age, ContactNo and EmailID attributes.

```
> db.Student.insertMany ([
    {RollNo: 10, Name: "ABC", Age: 20, ContactNo: 1112223334, EmailID: "CSE10.cs24@bmsce.ac.in"},
    {RollNo: 11, Name: "DEF", Age: 20, ContactNo: 2223334445, EmailID: "CSE11.cs24@bmsce.ac.in"},
    {RollNo: 12, Name: "GHI", Age: 20, ContactNo: 3334445556, EmailID: "CSE12.cs24@bmsce.ac.in"},
    {RollNo: 13, Name: "JKL", Age: 20, ContactNo: 4445556667, EmailID: "CSE13.cs24@bmsce.ac.in"}
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('693ff9c98e185b767bf90899'),
      '1': ObjectId('693ff9c98e185b767bf9089a'),
      '2': ObjectId('693ff9c98e185b767bf9089b'),
      '3': ObjectId('693ff9c98e185b767bf9089c')
    }
  }
```

Queries(Questions & Answers)

1. Write a query to update EmailID of a student with RollNo 10.

```
> db.Student.updateOne(
    {RollNo: 10},
    {$set: {EmailID: "CSE10Updated.cs24@bmsce.ac.in"}}
  );
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

2. Replace the student's name from "DEF" to "FEM" of RollNo 11.

```
> db.Student.updateOne (
    {RollNo: 11},
    {$set: {Name: "FEM"}}
  );
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

3. Export the created table into Local File System.

```
C:\Users\Sudhanva S M>mongoexport --db=Student --collection=Student --type=csv --fields=RollNo,Name,Age,ContactNo,EmailI
D --out=students.csv
2025-12-15T17:38:33.585+0530    connected to: mongodb://localhost/
2025-12-15T17:38:33.588+0530    exported 4 records
```

Or

a) Open your Collections Table.
b) Click on Export Data -> Export Full Collection.
c) Export File Type -> CSV.
d) A dialog box will open. Choose file destination and click save.

	A	B	C	D	E
1	RollNo	Name	Age	ContactNo	EmailID
2	10	ABC	20	1112223334	CSE10Updated.cs24@bmsce.ac.in
3	11	FEM	20	2223334445	CSE11.cs24@bmsce.ac.in
4	12	GHI	20	3334445556	CSE12.cs24@bmsce.ac.in
5	13	JKL	20	4445556667	CSE13.cs24@bmsce.ac.in

4. Drop the table.

```
> db.Student.drop();
< true
```

5. Import a given a CSV dataset from your Local File System into MongoDB collection.
   a) Open your collections table.
   b) Click Add Data → Import JSON or CSV file.
   c) Select previously saved CSV file (students.csv).
   d) Click Import.

Experiment 9: NoSQL – Customer Database

Specification of NoSQL – Customer Database Application:

The NoSQL customer database is designed to store and manage customer account information using a document-based data model in MongoDB. Each customer record is uniquely identified by a customer ID and includes attributes such as account balance and account type. The database must support insertion of multiple customer records and enable querying of customers whose total account balance exceeds a specified value for a given account type. It should also allow aggregation operations to determine the minimum and maximum account balances for each customer. Additionally, the system must support administrative operations such as exporting the customer collection to the local file system for backup purposes, importing customer data from external CSV files, and safely dropping collections when required. The database should demonstrate efficient data storage, flexible querying, and basic aggregation capabilities of NoSQL databases.

Create Database:

```
> use customer
< switched to db customer
```

Insert Appropriate Values:

Insert at least 4 documents each with Cust_id, Acc_Bal, Acc_Type attributes.

```
> db.customer.insertMany ([
    {Cust_id: 101, Acc_Bal: 1234, Acc_Type: "Z"},
    {Cust_id: 101, Acc_Bal: 777, Acc_Type: "Y"},
    {Cust_id: 103, Acc_Bal: 1634, Acc_Type: "Z"},
    {Cust_id: 104, Acc_Bal: 987, Acc_Type: "Z"},
    {Cust_id: 104, Acc_Bal: 1009, Acc_Type: "Y"}
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6940081e104a74526bf2021c'),
      '1': ObjectId('6940081e104a74526bf2021d'),
      '2': ObjectId('6940081e104a74526bf2021e'),
      '3': ObjectId('6940081e104a74526bf2021f'),
      '4': ObjectId('6940081e104a74526bf20220')
    }
  }
```

Queries:

1. Write a query to display those records whose total account balance is greater than 1200 of account type ‘Z’ for each Cust_id.

```
> db.customer.aggregate ([
    {$match: {Acc_Type: 'Z'}},
    {$group: {
      _id: "$Cust_id",
      total_balance: {$sum: "$Acc_Bal"}
    }},
    {$match: {total_balance: {$gt: 1200}}}
  ]);
< {
    _id: 101,
    total_balance: 1234
  }
  {
    _id: 103,
    total_balance: 1634
  }
```

2. Determine Minimum and Maximum account balance for each customer_id.

```
> db.customer.aggregate([
    {$group: {
        _id: "$Cust_id",
        min_balance: { $min: "$Acc_Bal" },
        max_balance: { $max: "$Acc_Bal" }
      }
    }
  ]);
< {
    _id: 104,
    min_balance: 987,
    max_balance: 1009
  }
  {
    _id: 101,
    min_balance: 777,
    max_balance: 1234
  }
  {
    _id: 103,
    min_balance: 1634,
    max_balance: 1634
  }
```

3. Export the created table into Local File System.

```
C:\Users\Sudhanva S M>mongoexport --db=customer --collection=customer --type=csv --fields=Cust_id,Acc_Bal,Acc_Type --out
=customer.csv
2025-12-15T18:41:49.137+0530    connected to: mongodb://localhost/
2025-12-15T18:41:49.141+0530    exported 5 records
```

Or

e) Open your Collections Table.
f) Click on Export Data -> Export Full Collection.
g) Export File Type -> CSV.
h) A dialog box will open. Choose file destination and click save.

	A	B	C
1	Cust_id	Acc_Bal	Acc_Type
2	101	1234	Z
3	101	777	Y
4	103	1634	Z
5	104	987	Z
6	104	1009	Y

4. Drop the table.

```
> db.customer.drop()
< true
```

5. Import a given a CSV dataset from your Local File System into MongoDB collection.
   a) Open your collections table.
   b) Click Add Data → Import JSON or CSV file.
   c) Select previously saved CSV file (customer.csv).
   d) Click Import.

Experiment 10: NoSQL – Restaurant Database

Specification of NoSQL – Restaurant Database Application:

The NoSQL restaurant database is designed to store and manage restaurant-related information using a document-oriented data model in MongoDB. Each restaurant document contains details such as restaurant ID, name, location information, cuisine type, and inspection scores. The database must support retrieval of all restaurant records and allow sorting of restaurant data based on specified attributes. It should enable selective querying to extract specific fields such as restaurant ID, name, town, and cuisine for restaurants meeting given score criteria. Additionally, the system must support aggregation operations to compute the average inspection score for each restaurant and perform pattern-based queries to identify restaurants located in areas with specific zip code formats. The database demonstrates flexible querying, sorting, filtering, and aggregation capabilities of MongoDB for real-world data analysis.

Create Database:

```
> use restaurant
< switched to db restaurant
```

Insert Appropriate Values:

Insert at least 4 documents each with Restaurant_id, Name, Cuisine, Address (Town, Zipcode), Score attributes.

```
> db.restaurant_details.insertOne([
    {restaurant_id: 1001,
     name: "Saffron & Sage",
     cuisine: "Modern Indian Fusion",
     address: {
       town: "Udaipur",
       zipcode: "101028"
     },
     score: 10.0
    }
  ]);
< {
    acknowledged: true,
    insertedId: ObjectId('69401559c7b448b946f298c3')
  }
```

```
> db.restaurant_details.insertOne([
    {restaurant_id: 1002,
     name: "The Gilded Noodle",
     cuisine: "Pan-Asian/Thai",
     address: {
       town: "Indore",
       zipcode: "452009"
     },
     score: 7.1
    }
  ]);
< {
    acknowledged: true,
    insertedId: ObjectId('69401566c7b448b946f298c4')
  }
```

```
> db.restaurant_details.insertOne([
    {restaurant_id: 1003,
     name: "Misty Malabar",
     cuisine: "South-Indian (Coastal)",
     address: {
       town: "Puducherry",
       zipcode: "105357"
     },
     score: 8.5
    }
  ]);
< {
    acknowledged: true,
    insertedId: ObjectId('6940156fc7b448b946f298c5')
  }
```

```
> db.restaurant_details.insertOne([
    {restaurant_id: 1004,
     name: "Velvet Vineyards",
     cuisine: "French-Italian",
     address: {
       town: "Mumbai",
       zipcode: "400001"
     },
     score: 18.5
    }
  ]);
< {
    acknowledged: true,
    insertedId: ObjectId('69401577c7b448b946f298c6')
  }
```

```
> db.restaurant_details.insertOne([
    {restaurant_id: 1005,
     name: "Indigo Masala",
     cuisine: "Indo-Mexican Fusion",
     address: {
       town: "Hyderabad",
       zipcode: "500108"
     },
     score: 12.5
    }
  ]);
< {
    acknowledged: true,
    insertedId: ObjectId('69401583c7b448b946f298c7')
  }
```

Queries:

1. Write a MongoDB query to display all the documents in the collection restaurants.

```
> db.restaurant_details.find();
< {
    _id: ObjectId('69401559c7b448b946f298c3'),
    restaurant_id: 1001,
    name: 'Saffron & Sage',
    cuisine: 'Modern Indian Fusion',
    address: {
      town: 'Udaipur',
      zipcode: '101028'
    },
    score: 10
  }
```

```
{
  _id: ObjectId('69401566c7b448b946f298c4'),
  restaurant_id: 1002,
  name: 'The Gilded Noodle',
  cuisine: 'Pan-Asian/Thai',
  address: {
    town: 'Indore',
    zipcode: '452009'
  },
  score: 7.1
}
```

```
{
  _id: ObjectId('6940156fc7b448b946f298c5'),
  restaurant_id: 1003,
  name: 'Misty Malabar',
  cuisine: 'South-Indian (Coastal)',
  address: {
    town: 'Puducherry',
    zipcode: '105357'
  },
  score: 8.5
}
```

```
{
  _id: ObjectId('69401577c7b448b946f298c6'),
  restaurant_id: 1004,
  name: 'Velvet Vineyards',
  cuisine: 'French-Italian',
  address: {
    town: 'Mumbai',
    zipcode: '400001'
  },
  score: 18.5
}
```

```
{
  _id: ObjectId('69401583c7b448b946f298c7'),
  restaurant_id: 1005,
  name: 'Indigo Masala',
  cuisine: 'Indo-Mexican Fusion',
  address: {
    town: 'Hyderabad',
    zipcode: '500108'
  },
  score: 12.5
}
```

2. Write a MongoDB query to arrange the name of the restaurants in descending along with all the columns.

```
> db.restaurant_details.find().sort({"name": -1});
< {
    _id: ObjectId('69401577c7b448b946f298c6'),
    restaurant_id: 1004,
    name: 'Velvet Vineyards',
    cuisine: 'French-Italian',
    address: {
      town: 'Mumbai',
      zipcode: '400001'
    },
    score: 18.5
  }
```

```
{
  _id: ObjectId('69401566c7b448b946f298c4'),
  restaurant_id: 1002,
  name: 'The Gilded Noodle',
  cuisine: 'Pan-Asian/Thai',
  address: {
    town: 'Indore',
    zipcode: '452009'
  },
  score: 7.1
}
```

```
< {
    _id: ObjectId('69401559c7b448b946f298c3'),
    restaurant_id: 1001,
    name: 'Saffron & Sage',
    cuisine: 'Modern Indian Fusion',
    address: {
      town: 'Udaipur',
      zipcode: '101028'
    },
    score: 10
  }
```

```
{
  _id: ObjectId('6940156fc7b448b946f298c5'),
  restaurant_id: 1003,
  name: 'Misty Malabar',
  cuisine: 'South-Indian (Coastal)',
  address: {
    town: 'Puducherry',
    zipcode: '105357'
  },
  score: 8.5
}
```

```
{
  _id: ObjectId('69401583c7b448b946f298c7'),
  restaurant_id: 1005,
  name: 'Indigo Masala',
  cuisine: 'Indo-Mexican Fusion',
  address: {
    town: 'Hyderabad',
    zipcode: '500108'
  },
  score: 12.5
}
```

3. Write a MongoDB query to find the restaurant Id, name, town and cuisine for those restaurants which achieved a score which is not more than 10.

```
> db.restaurant_details.find({score: {$lte: 10}});
< {
    _id: ObjectId('69401559c7b448b946f298c3'),
    restaurant_id: 1001,
    name: 'Saffron & Sage',
    cuisine: 'Modern Indian Fusion',
    address: {
      town: 'Udaipur',
      zipcode: '101028'
    },
    score: 10
  }
```

```
{
  _id: ObjectId('69401566c7b448b946f298c4'),
  restaurant_id: 1002,
  name: 'The Gilded Noodle',
  cuisine: 'Pan-Asian/Thai',
  address: {
    town: 'Indore',
    zipcode: '452009'
  },
  score: 7.1
}
```

```
{
  _id: ObjectId('6940156fc7b448b946f298c5'),
  restaurant_id: 1003,
  name: 'Misty Malabar',
  cuisine: 'South-Indian (Coastal)',
  address: {
    town: 'Puducherry',
    zipcode: '105357'
  },
  score: 8.5
}
```

4. Write a MongoDB query to find the average score for each restaurant.

```
> db.restaurant_details.aggregate([
    {
      $group: {
        _id: "$restaurant_id",
        avg_score: { $avg: "$score" }
      }
    }
  ]);
```

```
{
  _id: 1005,
  avg_score: 12.5
}
{
  _id: 1003,
  avg_score: 8.5
}
{
  _id: 1001,
  avg_score: 10
}
{
  _id: 1002,
  avg_score: 7.1
}
{
  _id: 1004,
  avg_score: 18.5
}
```

5. Write a MongoDB query to find the name and address of the restaurants that have a zip code that starts with '10'.

```
> db.restaurant_details.find(
    { "address.zipcode": { $regex: "^10" } },
    { name: 1, address: 1, _id: 0 }
  );
```

LEET Code Practice

1) Recyclable and low fat products. Write a solution to find the ids of products that are both low fat and recyclable.

```
SELECT product_id

FROM Products

WHERE low_fats = 'Y'

  AND recyclable = 'Y';
```

Accepted Runtime: 193 ms

Case 1

Input

Products =

```
| product_id | low_fats | recyclable |
| ---------- | -------- | ---------- |
| 0          | Y        | N          |
| 1          | Y        | Y          |
| 2          | N        | Y          |
| 3          | Y        | Y          |
| 4          | N        | N          |
```

Output

```
| product_id |
| ---------- |
| 1          |
| 3          |
```

2)Find Customer Referee. Find the names of the customer that are either:

i) referred by any customer with id != 2.

ii) not referred by any customer.

```
SELECT name

FROM Customer

WHERE referee_id != 2

   OR referee_id IS NULL;
```

Accepted Runtime: 78 ms

Case 1

Input

Customer =

```
| id | name | referee_id |
| -- | ---- | ---------- |
| 1  | Will | null       |
| 2  | Jane | null       |
| 3  | Alex | 2          |
| 4  | Bill | null       |
| 5  | Zack | 1          |
| 6  | Mark | 2          |
```

Output

```
| name |
| ---- |
| Will |
| Jane |
| Bill |
| Zack |
```

3) Product Sales Analysis I. Write a solution to report the product_name, year, and price for each sale_id in the Sales table.

```
SELECT
  p.product_name,
  s.year,
  s.price
FROM Sales s
JOIN Product p
 ON s.product_id = p.product_id;
```

Accepted Runtime: 91 ms

Case 1

Input

Sales =

```
| sale_id | product_id | year | quantity | price |
| ------- | ---------- | ---- | -------- | ----- |
| 1       | 100        | 2008 | 10       | 5000  |
| 2       | 100        | 2009 | 12       | 5000  |
| 7       | 200        | 2011 | 15       | 9000  |
```

Product =

```
| product_id | product_name |
| ---------- | ------------ |
| 100        | Nokia        |
| 200        | Apple        |
| 300        | Samsung      |
```

Output

```
| product_name | year | price |
| ------------ | ---- | ----- |
| Nokia        | 2009 | 5000  |
| Nokia        | 2008 | 5000  |
| Apple        | 2011 | 9000  |
```

4) Percentage of Users Attended a Contest. Write a solution to find the percentage of the users registered in each contest rounded to two decimals. Return the result table ordered by percentage in descending order. In case of a tie, order it by contest_id in ascending order.

```
SELECT

  r.contest_id,

  ROUND(

    COUNT(DISTINCT r.user_id) * 100.0 /

    (SELECT COUNT(*) FROM Users),

    2

  ) AS percentage

FROM Register r
```

GROUP BY r.contest_id

ORDER BY percentage DESC, r.contest_id ASC;

Accepted Runtime: 122 ms

Case 1

Input

Users =

```
| user_id | user_name |
| ------- | --------- |
| 6       | Alice     |
| 2       | Bob       |
| 7       | Alex      |
```

Register =

```
| contest_id | user_id |
| ---------- | ------- |
| 215        | 6       |
| 209        | 2       |
| 208        | 2       |
| 210        | 6       |
| 208        | 6       |
| 209        | 7       |
```

Output

```
| contest_id | percentage |
| ---------- | ---------- |
| 208        | 100        |
| 209        | 100        |
| 210        | 100        |
| 215        | 66.67      |
| 207        | 33.33      |
```

5) Biggest Single Number. A single number is a number that appeared only once in the MyNumbers table. Find the largest single number. If there is no single number, report null.

```
SELECT MAX(num) AS num

FROM (

    SELECT num

    FROM MyNumbers

    GROUP BY num

    HAVING COUNT(num) = 1

) t;
```

Accepted Runtime: 96 ms

Case 1 Case 2

Input

MyNumbers =

num
8
8
3
3
1
4

Output

num
6